8

PART 2: FRAMEWORK

From specification to software

What does the complete workflow from specification to deployed software look like in practice?

Riverside Clinics sets one go-live date for its booking feature: the Monday its marketing team emails twelve thousand patients that they can now book online. Two engineering teams are building toward that date. One is the team this book has followed since Chapter 1, the team that spent six chapters turning a one-line request into a vision, requirements, a use case, a domain model, an architecture, and an implementation skill. The other belongs to a physiotherapy group Riverside acquired six weeks earlier, and it received the same one-line request Riverside's own team got at the very start: let patients book appointments online, nothing else specified. Both teams have the same AI assistants. Both teams are competent engineers. What they don't have, going into the same six weeks, is the same amount of work already done.

CHAPTER PROMISE

By the end of this chapter, you can run a feature from a finished specification to a deployed release using the same ten-stage workflow, and tell which of its stages tolerates a shortcut and which one never does.

Reading time: 13 minutes

What the same deadline actually measured

The acquired team opens an assistant on day one and has a working prototype by lunch. It looks like a head start. Doctors can add themselves, patients can pick a time, appointments save to a database. By week three, the prototype has been rebuilt four times. Doctors can double-book patients. Nobody on the team can say with confidence whether a patient with two email addresses on file is one person or two, because nobody decided, and the assistant decided for them, differently, in two different files. Every fix it produces is correct on its own terms and breaks something the last fix depended on.

Riverside's own team opens an assistant on day one too, and spends most of that day reading rather than typing. It reviews the specification Chapter 5 assembled, updates one domain-model detail a stakeholder conversation flagged as stale, and asks the assistant to implement a single use case against an existing domain model, architecture, and skill. Four days later, the feature is in review. The rest of the six weeks goes to two more use cases and to hardening what already works.

Both teams are running the same model, from the same vendor, on the same day. The four days and the three-week spiral measure something else entirely: how much of the business decision was settled before either team typed a prompt. Chapters 2 through 7 spent their effort building that settlement, one layer at a time: a vision, requirements, a use case, a domain model, an architecture, a skill. This chapter is about what happens once all six exist, and a team still has to turn them into a feature that runs in production by Monday.

The workflow that replaces improvising

Having a specification turns delivery into an ordered sequence rather than a single prompt: ten stages, each answering a question the stage before it couldn't, each removing a decision an AI assistant would otherwise have to invent on its own.

1. Vision: why the feature exists, and for whom
2. Requirements: what the system must do, observably
3. Use case: the complete interaction, step by step
4. Domain model: the business objects the feature runs on
5. Architecture: the structural rules the feature must fit
6. Skill: the implementation conventions the team already has
7. Implementation: the assistant translates stages 1 through 6 into code
8. Testing: the code is checked against what the use case actually said
9. Review: a human checks business correctness and consistency
10. Deployment: the feature ships through the pipeline that already exists

Chapters 2 through 7 built the first six stages for Riverside's booking feature. Most of that work does not repeat itself for every new feature; a vision, requirements, a domain model, an architecture, and a skill are written once and reused across the many use cases written against them. What does repeat, every time a team turns a use case into working software, is the last four stages, implementation, testing, review, and deployment, plus one step this chapter adds before any of them, confirming the first six stages are still current.

> **The stages that take the longest happen once. The stages that happen every time are the ones a team can now run in days instead of weeks.**

Running the workflow once a specification exists

This is the cycle a team runs for every use case, once the vision, requirements, domain model, architecture, and skill it depends on are already written. It is short enough to run in days, and none of its five steps is optional.

1. Review the specification for staleness before implementing anything. Confirm the vision, requirements, use case, domain model, architecture, and skill still match what stakeholders agreed most recently, and update whichever one doesn't.
2. Ask the assistant to implement one use case against the existing domain model, architecture, and skill, and nothing beyond them. A specification-shaped request stays short because the decisions it depends on already live somewhere the assistant can read.
3. Generate tests from the use case's own scenarios: one test per step in the main flow, one test per alternative flow, so the tests and the specification describe the same behavior rather than two different ones.
4. Route the implementation to a human reviewer whose time is set by business risk, not by how many lines the assistant returned.
5. Deploy through the pipeline that already exists. Nothing about this workflow changes what continuous integration and delivery do once code reaches them.

Riverside Clinics: from specification review to working code

Here is Riverside's team running the first two steps of that cycle against the booking specification Chapters 2 through 7 built.

Step 1: specification review

```
The 90-day booking horizon from Chapter 3 hasn't changed. The domain model
has: clinic operations now lets a doctor block part of a day, not only a whole
one, and the model Chapter 4 wrote has nowhere to record a block that short.
The team adds one value object for it before writing anything else. Everything
else in the specification, the vision, the use case, the architecture, the
Spring Boot skill, is confirmed current and left untouched.
```

Step 2: implementation

```
"Implement the Schedule Appointment use case (Chapter 3) against the Riverside
Clinics domain model (Chapter 4, plus the new value object for partial-day
blocks) and the Spring Boot skill (Chapter 7)."
```

```
The request is one sentence. Minutes later, the feature exists: a booking
endpoint, a persistence layer, validation against the 90-day window and the
new partial-day rule, a confirmation notification.
```

Nothing in these two steps required the team to explain what a doctor is, what counts as an eligible time slot, or how Riverside logs an error. Chapters 3, 4, and 7 already answered those questions, once, for every feature that comes after this one.

Riverside Clinics: from generated tests to a deployed feature

Step 3: test generation

Each step of the use case's main scenario becomes a test. "System re-checks that the slot is open, then reserves it" becomes a test that a conflicting appointment is rejected. One alternative flow, a selected slot no longer available, becomes a test that the system offers another selection rather than failing silently. The tests describe the same behavior the use case describes, because they were generated from it rather than from the code.

Step 4: human review

Simon Martinelli, whose specification-driven methodology this book draws on throughout, puts the standard plainly: review effort should scale with business risk, not with how much code the assistant produced. Booking touches a patient's schedule but not billing or a clinical decision, so the reviewer spends little time on the routine persistence wiring and most of it on the one case that matters here: two patients reaching for the same slot at once, and whether the confirmation that goes out afterward is the one that should have.

Step 5: deployment

The feature ships through the continuous integration and delivery pipeline Riverside already runs, unchanged by anything in this chapter. Deployment is where the four preceding steps prove they were done correctly.

Where the workflow breaks down

▲ **WARNING**

The methodology's own account goes further and claims that once implementation is fast, regenerating a whole feature from an updated specification becomes as routine as re-running a build, cheap enough that code stops being something a team protects and becomes something it simply reproduces. That is a forward-looking possibility, not a settled fact. Nothing in the Riverside case, and nothing in the case studies this book draws on, shows implementation time and regeneration time actually converging in practice. This chapter treats it as a question worth watching, rather than a plan to build a team's process around today.

Skipping specification review because the specification already exists	A team implements confidently against a domain model that's a version behind the business, and the resulting bug looks like a coding mistake when it was actually a stale rule nobody checked.
Reviewing every feature at the same intensity, regardless of what's at stake	A reporting screen gets the same scrutiny as a billing change, and the reviewer has no time left when a risky feature needs it most.

Field guide: the specification-to-software workflow checklist

- ☐ Vision confirmed current, or updated if it drifted
- ☐ Requirements still observable, testable, and matched to what stakeholders agreed most recently
- ☐ Use case covers alternative flows, not only the main scenario
- ☐ Domain model reflects today's business rules, not the rules as they stood when it was written
- ☐ Architecture and skill unchanged, or the change is deliberate and recorded
- ☐ Implementation requested against the use case, domain model, architecture, and skill, nothing more
- ☐ Tests generated from the use case's own scenarios, one test per flow
- ☐ Review effort set by business risk, not by lines of generated code
- ☐ Deployment run through the existing pipeline, unchanged
- ☐ Specification updated to match, if anything changed during implementation

Run this before calling a feature done. A feature that passes every item but the last one will drift from its own specification by the time the next change reaches it.

What to remember

- The gap between a team that ships in days and a team that spirals for weeks is preparation, not which AI model either one used.
- A specification reaches software through ten stages: six that write the specification, and four that turn each use case into shipped software.
- Testing exists to confirm the implementation matches the specification, not merely that the code runs without error.
- Review effort should scale with business risk, not with how much code an assistant returned.
- A feature is done when its specification is current enough for someone else to read it and understand what shipped, and why.

QUESTIONS FOR YOUR TEAM

- The last time your team's review took longer than the implementation, was the feature actually high-risk, or did every feature get the same scrutiny by default?
- If your specification and your deployed feature disagreed today, which one would your team trust?

ONE ACTION TO TAKE NEXT

Take the next feature on your team's board that already has a specification behind it. Run it through the five-step cycle in this chapter, in order, and note which step your team was already skipping.

TRANSITION. Every example so far has had someone who could still say what the business meant: a stakeholder to interview, a habit to write down, a convention already sitting in a README nobody had opened. Most engineers spend their careers on systems where nobody left can say, and the only description of what the software does is the software itself. The next chapter is about recovering one.

From specification to software ends here.